

لغة بايثون

Python Programming

ون

فهرس المحتويات

1 مقدمة عن لغة بايثون

2 تنصيب وتجهيز البيئة

3 أساسيات اللغة

4 أنواع البيانات

5 النصوص (Strings)

6 القوائم (Lists)

7 الصفوف (Tuples)

8 القواميس (Dictionaries)

9 المجموعات (Sets)

10 الجمل الشرطية وحلقات التكرار

11 الدوال (Functions)

12 دوال لامدا (Lambda)

13 الوحدات والمكتبات (Modules)

14 التعامل مع الملفات

15 معالجة الأخطاء (Exceptions)

16 البرمجة كائنية التوجه (OOP)

17 الوراثة (Inheritance)

18 المزخرفات (Decorators)

1. مقدمة عن لغة بايثون

بايثون (Python) هي لغة برمجة عالية المستوى، مفسَّرة (Interpreted)، ومتعددة الاستخدامات. طورها **Guido van Rossum** عام 1991. تتميز بايثون بتركيبها البسيط والواضح الذي يجعل قراءة الكود سهلة، مما جعلها من أكثر اللغات شعبية في العالم.

مميزات بايثون

- **بسيطة وسهلة التعلم** - تركيبها قريب من اللغة الإنجليزية
- **مفسَّرة (Interpreted)** - تنفذ الكود سطرًا سطرًا دون حاجة للترجمة
- **متعددة الاستخدامات** - ويب، ذكاء اصطناعي، تحليل بيانات، تطبيقات سطح المكتب
- **مكتبة ضخمة** - مكتبة قياسية كبيرة ومكتبات طرف ثالث لا تعد
- **منصات متعددة** - تعمل على Windows, Linux, macOS
- **مفتوحة المصدر** - مجانية تماماً
- **دعم المجتمع** - مجتمع كبير ونشط من المطورين

مجالات استخدام بايثون

- تطوير الويب (Django, Flask)
- علم البيانات والذكاء الاصطناعي (TensorFlow, PyTorch, scikit-learn)
- تحليل البيانات (Pandas, NumPy, Matplotlib)
- الأمن السيبراني والاختراق الأخلاقي
- تطبيقات سطح المكتب (PyQt, Tkinter)
- الأتمتة وكتابة السكريبتات
- تطوير الألعاب (Pygame)
- تطبيقات الشبكات والاتصالات

2. تنصيب وتجهيز البيئة

تنصيب بايثون

يمكن تحميل بايثون من الموقع الرسمي:

<https://www.python.org/downloads/>

التحقق من التنصيب

بعد التنصيب، افتح الطرفية (Terminal) واكتب:

```
python --version
Python 3.x.x

# أو
python3 --version
```

أدوات التطوير (IDEs)

- VS Code - مع إضافة Python
- PyCharm - بيئة متكاملة احترافية
- Jupyter Notebook - لتحليل البيانات
- IDLE - البيئة المدمجة مع بايثون

تشغيل أول برنامج

لنبدأ بأول برنامج تقليدي:

```
print("مرحباً بالعالم")
# Hello World بالعربية
```

لتشغيل الملف، احفظه باسم hello.py ثم نفذ:

```
python3 hello.py
```

3. أساسيات اللغة

المتغيرات (Variables)

في بايثون، لا حاجة لتحديد نوع المتغير. المتغير هو مجرد اسم يشير إلى قيمة في الذاكرة.

```
x = 5          # عدد صحيح
name = "Ahmed" # نص
pi = 3.14       # عدد عشري
is_active = True # قيمة منطقية
```

قواعد تسمية المتغيرات

- يبدأ بحرف أو شرطة سفلية _
- يحتوي على أحرف، أرقام، وشرطات سفلية
- حساس لحالة الأحرف (name ≠ Name)
- لا يمكن استخدام الكلمات المحجوزة (if, for, while, etc).

```
my_var = 10      # صحيح
myVar = 20        # صحيح أيضاً
_private = 30     # شرطة سفلية مسموحة
2things = 5       # خطأ! لا يبدأ برقم
```

التعليقات (Comments)

التعليقات أسطر يتم تجاهلها عند التنفيذ. تستخدم لشرح الكود.

```
# هذا تعليق بسطر واحد

"""
هذا تعليق
متعدد الأسطر
"""
```

الإدخال والإخراج (Input / Output)

```
# إخراج نص
print("Hello, World!")

# إدخال من المستخدم
```

```
name = input("ما اسمك؟ ")  
print("مرحباً", name)
```

ما اسمك؟ أحمد << مرحباً أحمد >>

4. أنواع البيانات (Data Types)

بايثون تدعم أنواع بيانات متعددة. إليك الأنواع الأساسية:

النوع	الفئة	مثال	الوصف
int	عدد صحيح	x = 10	الأعداد الصحيحة
float	عدد عشري	x = 3.14	الأعداد الكسرية
complex	عدد مركب	x = 2+3j	للمعاملات المركبة
str	نص	x = "Hello"	سلسلة محرفية
bool	منطقي	x = True	False أو True
list	قائمة	x = [1, 2, 3]	مصفوفة قابلة للتعديل
tuple	صف	x = (1, 2)	مصفوفة غير قابلة للتعديل
dict	قاموس	x = {"a": 1}	مفتاح وقيمة
set	مجموعة	x = {1, 2, 3}	عناصر فريدة غير مرتبة
NoneType	عدم	x = None	قيمة فارغة

التحقق من النوع

```
x = 10
print(type(x))    # <class 'int'>

y = "Hello"
print(type(y))    # <class 'str'>
```

تحويل بين الأنواع (Type Casting)

```
x = int("10")      # 10 (str → int)
y = float("3.14")  # 3.14 (str → float)
z = str(100)       # "100" (int → str)
b = bool(1)        # True (int → bool)
```


العمليات الحسابية

العملية	الرمز	مثال	النتيجة
جمع	+	3 + 5	8
طرح	-	3 - 5	2
ضرب	*	3 * 5	15
قسمة	/	2 / 5	2.5
قسمة صحيحة	//	2 // 5	2
باقي القسمة	%	2 % 5	1
أس	**	3 ** 2	8

5. النصوص (Strings)

النصوص في بايثون هي تسلسل من المحارف (Characters). يمكن تعريفها باستخدام علامتي اقتباس مفردة أو مزدوجة.

إنشاء النصوص

```
s1 = 'Hello'          # اقتباس مفرد
s2 = "World"          # اقتباس مزدوج
s3 = '''Multi
Line'''              # نص متعدد الأسطر
s4 = """ثلاث علامات
اقتباس"""
```

التنسيق (String Formatting)

طريقة 1: f-strings (الأفضل)

```
name = "Ahmed"
age = 25
print(f"سنة {age} وعمري {name} اسمي")
# وعمري 25 سنة Ahmed اسمي
```

طريقة 2: format ()

```
print("سنة {} وعمري {}".format(name, age))
```

طريقة 3: % (قديمة)

```
print("سنة %d وعمري %s اسمي" % (name, age))
```

عمليات شائعة على النصوص

```
s = "Hello Python"

len(s)          # طول النص - 12
s.upper()        # "HELLO PYTHON" - أحرف كبيرة
s.lower()        # "hello python" - أحرف صغيرة
s.capitalize()   # "Hello python" - أول حرف كبير
s.title()        # "Hello Python" - أول كل كلمة كبيرة
s.strip()        # يزيل المسافات من البداية والنهاية
s.split(" ")     # ['Hello', 'Python'] - تقطيع
s.join(["A", "B"]) # "A B" - ربط
s.replace("Hello", "Hi") # "Hi Python" - استبدال
s.find("Python")  # 6 - إيجاد موقع
```

```
s.startswith("He")    # True - تبدأ بـ
s.endswith("on")      # True - تنتهي بـ
s.count("l")          # 2 - عدد التكرارات
```

الاقطعاع (Slicing)

```
s = "Python Programming"
s[0]      # 'P' - أول حرف
s[-1]     # 'g' - آخر حرف
s[0:6]    # 'Python' - من 0 إلى 5
s[7:]     # 'Programming' - من 7 للنهاية
s[:6]     # 'Python' - من البداية إلى 5
s[::2]    # 'Pto rgamn' - خطوة 2
s[::-1]   # 'gnimmargorP nohtyP' - قلب النص
```

الحروف الهاربة (Escape Characters)

الرمز	المعنى
n\	سطر جديد
t\	تبويب (Tab)
\\	خط مائل عكسي
'\	اقتباس مفرد
"\	اقتباس مزدوج

6. القوائم (Lists)

القائمة هي مجموعة مرتبة من العناصر يمكن تعديلها. العناصر يمكن أن تكون من أي نوع.

إنشاء القوائم

```
empty = []           # قائمة فارغة
numbers = [1, 2, 3, 4, 5] # قائمة أعداد
mixed = [1, "Hello", 3.14, True] # قائمة مختلطة
nested = [[1, 2], [3, 4]] # قائمة متداخلة
```

عمليات القوائم

```
lst = [1, 2, 3, 4, 5]

lst.append(6)           # إضافة في النهاية - [1,2,3,4,5,6]
lst.insert(0, 0)        # إضافة في موقع - [0,1,2,3,4,5,6]
lst.remove(3)           # إزالة أول ظهور للقيمة 3
lst.pop()               # إزالة ويرجع آخر عنصر
lst.pop(2)              # إزالة ويرجع العنصر في الموقع 2
lst.clear()             # إفراغ القائمة
lst.index(4)            # يرجع موقع القيمة 4
lst.count(2)            # عدد مرات ظهور 2
lst.sort()              # ترتيب تصاعدي
lst.sort(reverse=True)  # ترتيب تنازلي
lst.reverse()           # عكس الترتيب
len(lst)                # عدد العناصر
sum(lst)                # مجموع العناصر
max(lst)                # أكبر قيمة
min(lst)                # أصغر قيمة
```

الاشتراك (List Slicing)

```
lst = [10, 20, 30, 40, 50, 60]
lst[0]           # 10
lst[-1]          # 60
lst[1:4]         # [20, 30, 40]
lst[:3]          # [10, 20, 30]
lst[::2]         # [10, 30, 50]
lst[::-1]        # [60, 50, 40, 30, 20, 10]
```

التكرار على القوائم

```
fruits = ["برتقال", "موز", "تفاح"]

for fruit in fruits:
    print(fruit)
```


7. الصفوف (Tuples)

الصف (Tuple) يشبه القائمة لكنه غير قابل للتعديل (Immutable). يستخدم للبيانات الثابتة.

```
# إنشاء صف
point = (3, 4)
rgb = (255, 128, 0)
single = (5,) # صف بعنصر واحد - الفاصلة مهمة!
mixed = (1, "Hello", 3.14)

# الوصول للعناصر
print(point[0]) # 3
x, y = point # Unpacking: x=3, y=4

# الصفوف غير قابلة للتعديل
point[0] = 10 # خطأ! TypeError

# دوال الصفوف
len(point) # 2
point.count(3) # 1
point.index(4) # 1
```

متى تستخدم Tuple بدلاً من List؟

- عندما تريد بيانات ثابتة لا تتغير
- عندما تريد استخدام البيانات كمفتاح في Dictionary
- عندما تريد أداء أسرع (Tuples أسرع من Lists)
- عندما تريد إرجاع قيم متعددة من دالة

8. القواميس (Dictionaries)

القاموس (Dict) هو مجموعة من المفاتيح (Keys) والقيم (Values). كل مفتاح فريد ولا يتكرر.

إنشاء القواميس

```
person = {  
    "name": "أحمد",  
    "age": 25,  
    "city": "القاهرة"  
}  
  
# استخدام dict()  
person2 = dict(name="Sara", age=30, city="Riyadh")
```

الوصول للعناصر

```
print(person["name"])      # أحمد  
print(person.get("name"))  # إذا المفتاح غير موجود None يرجع أحمد - آمن  
print(person.get("phone", "غير موجود")) # غير موجود - قيمة افتراضية
```

تعديل وإضافة عناصر

```
person["age"] = 26          # تعديل قيمة موجودة  
person["phone"] = "012345"  # إضافة مفتاح جديد  
person.update({"country": "مصر"}) # إضافة/تحديث متعدد
```

حذف عناصر

```
del person["phone"]        # حذف مفتاح  
person.pop("city")         # حذف ويرجع القيمة  
person.popitem()           # حذف آخر مفتاح مضاف  
person.clear()             # إفريغ القاموس
```

التكرار على القواميس

```
for key in person:          # المفاتيح فقط  
    print(key)  
  
for key, value in person.items(): # المفتاح والقيمة  
    print(f"{key}: {value}")
```

```
for key in person.keys():           # المفاتيح (مراجعة)
    print(key)

for val in person.values():         # القيم فقط
    print(val)
```


9. المجموعات (Sets)

المجموعة (Set) هي مجموعة غير مرتبة من العناصر الفريدة. تستخدم للعمليات الرياضية على المجموعات.

```
# إنشاء مجموعة
a = {1, 2, 3, 4, 5}
b = set([3, 4, 5, 6, 7]) # من قائمة
empty = set()           # مجموعة فارغة ({} تنشئ قاموساً فارغاً)

# العمليات الأساسية
a.add(6)                 # إضافة عنصر
a.remove(6)              # حذف عنصر (خطأ إن لم يوجد)
a.discard(10)            # حذف عنصر (بدون خطأ إن لم يوجد)
a.pop()                  # يزيل ويرجع عنصراً عشوائياً
a.clear()                # إفريغ

# العمليات الرياضية
a | b                    # اتحاد (Union)
a & b                    # تقاطع (Intersection)
a - b                    # الفرق (Difference)
a ^ b                    # الفرق المتماثل (Symmetric Diff)
a.union(b)               # اتحاد
a.intersection(b)        # تقاطع
a.difference(b)          # فرق
```

10. الجمل الشرطية وحلقات التكرار

الجمل الشرطية (if / elif / else)

```
x = 10

if x > 0:
    print("العدد موجب")
elif x == 0:
    print("العدد صفر")
else:
    print("العدد سالب")
```

عوامل المقارنة

العامل	المعنى	مثال
==	يساوي	x == 5
!=	لا يساوي	x != 5
<	أكبر من	x > 5
>	أصغر من	x < 5
<=	أكبر أو يساوي	x >= 5
>=	أصغر أو يساوي	x <= 5

العوامل المنطقية

العامل	المعنى	مثال
and	و - الشرطان صحيحان	x > 0 and x < 10
or	أو - أحد الشرطين صحيح	x > 0 or x < -10
not	عكس القيمة المنطقية	not x > 0

حلقة for

```
# التكرار على قائمة
fruits = ["تفاح", "موز", "عنب"]
for fruit in fruits:
    print(fruit)

# range() - تكرار رقمي
for i in range(5):          # 0, 1, 2, 3, 4
    print(i)

for i in range(2, 8):       # 2, 3, 4, 5, 6, 7
    print(i)

for i in range(0, 10, 2):   # 0, 2, 4, 6, 8
    print(i)
```

حلقة while

```
count = 0
while count < 5:
    print(count)
    count += 1          # count = count + 1

# حلقة لا نهائية (توقف بشرط)
while True:
    cmd = input("أدخل أمر: ")
    if cmd == "exit":
        break
```

break, continue, pass

```
# break - يخرج من الحلقة
for i in range(10):
    if i == 5:
        break          # يتوقف عند 5
    print(i)

# continue - يتجاوز التكرار الحالي
for i in range(5):
    if i == 2:
        continue       # يتخطى 2
    print(i)

# pass - عنصر نائب (يفعل شيئاً)
if x > 0:
    pass               # سنكتب الكود لاحقاً
```

11. الدوال (Functions)

الدالة هي كتلة من الكود تنفذ مهمة محددة. تستخدم لإعادة استخدام الكود وتنظيمه.

تعريف دالة

```
def greet():  
    print("مرحباً بالعالم!")  
  
# استدعاء الدالة  
greet() # مرحباً بالعالم!
```

دوال بمعاملات (Parameters)

```
def greet(name):  
    print(f"مرحباً {name}!")  
  
greet("أحمد") # مرحباً أحمد!  
  
# معاملات متعددة  
def add(a, b):  
    return a + b  
  
result = add(5, 3) # 8
```

قيم افتراضية للمعاملات

```
def greet(name, greeting="مرحباً"):   
    print(f"{greeting} {name}!")  
  
greet("أحمد") # مرحباً أحمد!  
greet("أحمد", "أهلاً") # أهلاً أحمد!
```

معاملات الكلمة المفتاحية (Keyword Arguments)

```
def show_info(name, age, city):  
    print(f"{name}, {age} سنة، من {city}")  
  
show_info(age=25, city="القاهرة", name="أحمد") # الترتيب غير مهم
```

kwargs** و args*

args* - لاستقبال عدد غير محدد من المعاملات (ك Tuple)

```
def sum_all(*args):
    return sum(args)

print(sum_all(1, 2, 3, 4, 5)) # 15
```

kwargs** - لاستقبال عدد غير محدد من المعاملات المسماة (ك Dict)

```
def print_info(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

print_info(name="أحمد", age=25, job="مهندس")
```

نطاق المتغيرات (Scope)

```
x = 10 # متغير عام (Global)

def my_func():
    x = 5 # لا يؤثر على العام - متغير محلي (Local)
    print(x)

my_func()
print(x) # 10

# لتعديل المتغير العام داخل دالة
def change_global():
    global x
    x = 20

change_global()
print(x) # 20
```

التوثيق (Docstrings)

```
def add(a, b):
    """
    . ترجع مجموع عددين

    Args:
        a: العدد الأول
        b: العدد الثاني

    Returns:
        a و b مجموع
    """
    return a + b

# عرض التوثيق
print(add.__doc__)
help(add)
```

12. دوال لامدا (Lambda Functions)

دوال مجهولة صغيرة تكتب في سطر واحد. تستخدم عادة مع دوال مثل `map`, `filter`, `sort`.

```
# دالة لامدا بسيطة
square = lambda x: x ** 2
print(square(5))          # 25

# لامدا بمعاملين
add = lambda a, b: a + b
print(add(3, 4))          # 7

# استخدام map()
nums = [1, 2, 3, 4, 5]
squared = list(map(lambda x: x**2, nums))
# [1, 4, 9, 16, 25]

# استخدام filter()
evens = list(filter(lambda x: x % 2 == 0, nums))
# [2, 4]

# استخدام sorted()
students = [("أحمد", 85), ("سارة", 92), ("خالد", 78)]
sorted_students = sorted(students, key=lambda s: s[1], reverse=True)
# [("92", "سارة"), ("85", "أحمد"), ("78", "خالد")]
```

13. الوحدات والمكتبات (Modules)

الوحدة (Module) هي ملف بايثون يحتوي على دوال ومتغيرات. المكتبة (Package) هي مجموعة من الوحدات.

استيراد الوحدات

```
# استيراد وحدة كاملة
import math
print(math.pi)          # 3.141592653589793
print(math.sqrt(16))    # 4.0

# استيراد دوال محددة
from math import pi, sqrt
print(sqrt(25))         # 5.0

# استيراد باختصار
import math as m
print(m.pi)             # 3.14159

# استيراد كل شيء
from math import *      # غير مستحسن
```

المكتبة القياسية - وحدات هامة

الوصف	الوحدة
دوال رياضية متقدمة	math
توليد أرقام عشوائية	random
التعامل مع التاريخ والوقت	datetime
التفاعل مع نظام التشغيل	os
معلومات ومتغيرات النظام	sys
التعامل مع ملفات JSON	json
التعبيرات المنتظمة (Regex)	re
هياكل بيانات إضافية	collections
دوال للتكرار المتقدم	itertools

أمثلة على الوحدات

```
# random - أرقام عشوائية
import random
print(random.randint(1, 10))      # عدد عشوائي بين 1 و 10
print(random.choice(["أ", "ب", "ج"])) # اختيار عشوائي
random.shuffle(lst)               # خلط القائمة

# datetime - التاريخ والوقت
from datetime import datetime, timedelta
now = datetime.now()
print(now.strftime("%Y-%m-%d %H:%M:%S"))
tomorrow = now + timedelta(days=1)

# os - نظام التشغيل
import os
print(os.getcwd())                # المسار الحالي
os.listdir(".")                  # محتويات المجلد
os.mkdir("new_folder")           # إنشاء مجلد
```

إنشاء وحدة خاصة

أنشئ ملفاً باسم `mymodule.py`:

```
# mymodule.py
PI = 3.14159

def add(a, b):
    return a + b

class Calculator:
    def multiply(self, a, b):
        return a * b
```

ثم استخدمها في ملف آخر:

```
# main.py
import mymodule
print(mymodule.PI)
print(mymodule.add(5, 3))
calc = mymodule.Calculator()
print(calc.multiply(4, 5))
```


14. التعامل مع الملفات

قراءة ملف

```
# فتح وقراءة الملف كاملاً
with open("file.txt", "r") as f:
    content = f.read()
    print(content)

# قراءة سطر سطر
with open("file.txt", "r") as f:
    for line in f:
        print(line.strip())

# قراءة كقائمة أسطر
with open("file.txt", "r") as f:
    lines = f.readlines()
```

كتابة ملف

```
# كتابة (مسح المحتوى السابق)
with open("file.txt", "w") as f:
    f.write("Hello, World!\n")
    f.write("السطر الثاني\n")

# إضافة إلى نهاية الملف
with open("file.txt", "a") as f:
    f.write("إضافة سطر\n")
```

أنماط فتح الملفات

النمط	الوصف
"r"	قراءة (افتراضي)
"w"	كتابة (مسح المحتوى)
"a"	إضافة إلى النهاية
"r+"	قراءة وكتابة
"b"	وضع ثنائي (Binary)

```
import json

# JSON تحويل قاموس إلى
data = {"name": "أحمد", "age": 25}
json_str = json.dumps(data, ensure_ascii=False)
print(json_str)

# لملف JSON كتابة
with open("data.json", "w") as f:
    json.dump(data, f, ensure_ascii=False, indent=4)

# من ملف JSON قراءة
with open("data.json", "r") as f:
    loaded = json.load(f)
    print(loaded["name"])
```

15. معالجة الأخطاء (Exceptions)

الأخطاء في بايثون تسمى استثناءات (Exceptions). يمكنك التعامل معها باستخدام try/except.

try / except / else / finally

```
try:
    x = int(input("أدخل رقم: "))
    result = 10 / x
    print(result)
except ValueError:
    print("خطأ: يجب إدخال رقم صحيح")
except ZeroDivisionError:
    print("خطأ: لا يمكن القسمة على صفر")
except Exception as e:
    print(f"حدث خطأ غير متوقع {e}")
else:
    print("تمت العملية بنجاح") # لن ينفذ إن لم يحدث خطأ
finally:
    print("هذا السطر ينفذ دائماً") # ينفذ في كل الأحوال
```

أنواع الأخطاء الشائعة

الخطأ	السبب
SyntaxError	خطأ في تركيب الكود
NameError	متغير غير معرف
TypeError	نوع غير مناسب لعملية
ValueError	قيمة غير مناسبة
IndexError	مؤشر خارج حدود القائمة
KeyError	مفتاح غير موجود في القاموس
ZeroDivisionError	قسمة على صفر
FileNotFoundError	ملف غير موجود
ImportError	فشل استيراد وحدة
AttributeError	خاصية أو دالة غير موجودة

رفع استثناء (Raise Exception)

```
def check_age(age):  
    if age < 0:  
        raise ValueError("العمر لا يمكن أن يكون سالباً")  
    if age < 18:  
        raise ValueError("يجب أن تكون أكبر من 18 سنة")  
    return True  
  
try:  
    check_age(-5)  
except ValueError as e:  
    print(e)
```

إنشاء استثناء مخصص

```
class CustomError(Exception):  
    def __init__(self, message):  
        super().__init__(message)  
        self.message = message  
  
raise CustomError("هذا خطأ مخصص")
```

16. البرمجة كائنية التوجه (OOP)

البرمجة كائنية التوجه (OOP) هي نمط برمجة يعتمد على الكائنات (Objects) التي تحتوي على بيانات (Attributes) وسلوك (Methods).

الفئة والكائن (Class & Object)

```
class Student:
    # دالة البناء (Constructor)
    def __init__(self, name, age):
        self.name = name      # خاصية (Attribute)
        self.age = age
        self.grades = []

    # دالة (Method)
    def add_grade(self, grade):
        self.grades.append(grade)

    def average(self):
        if not self.grades:
            return 0
        return sum(self.grades) / len(self.grades)

    def __str__(self):
        return f"طالب: {self.name}, العمر: {self.age}"

# إنشاء كائنات
s1 = Student("أحمد", 20)
s2 = Student("سارة", 22)

# استخدام الكائنات
s1.add_grade(85)
s1.add_grade(90)
s1.add_grade(78)
print(f"{s1.name} - المعدل: {s1.average():.2f}")
```

خصائص الفئة والكائن

```
class Car:
    # خاصية على مستوى الفئة (Class Attribute)
    wheels = 4

    def __init__(self, brand, model):
        # خاصية على مستوى الكائن (Instance Attribute)
        self.brand = brand
        self.model = model
        self.__private = "سر"  # خاصية خاصة (Name Mangling)

car1 = Car("Toyota", "Camry")
print(car1.brand)      # Toyota
print(Car.wheels)      # 4
print(car1.wheels)     # 4
# print(car1.__private)  # خطأ!
```

الدوال الخاصة (Magic Methods / Dunder Methods)

الوصف	الدالة
البناء (Constructor)	__init__
تمثيل نصي للكائن (للمستخدم)	__str__
تمثيل نصي للكائن (للمطور)	__repr__
طول الكائن	__len__
جمع كائنين (+)	__add__
المقارنة (==)	__eq__
أصغر من (>)	__lt__
الوصول بالمؤشر ([])	__getitem__

17. الوراثة (Inheritance)

الوراثة تسمح لفئة (Subclass/Child) باستخدام خصائص ودوال فئة أخرى (Parent/Superclass).

وراثة بسيطة

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        raise NotImplementedError("يجب تطبيق هذه الدالة")

class Dog(Animal):
    def speak(self):
        return "Woof!"

class Cat(Animal):
    def speak(self):
        return "Meow!"

dog = Dog("Buddy")
cat = Cat("Kitty")
print(f"{dog.name} says {dog.speak()}")
print(f"{cat.name} says {cat.speak()}")
```

super () - استدعاء الفئة الأب

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def introduce(self):
        return f"سنة {self.age} وعمرى {self.name} أنا"

class Employee(Person):
    def __init__(self, name, age, job, salary):
        super().__init__(name, age) # استدعاء بناء الفئة الأب
        self.job = job
        self.salary = salary

    def introduce(self): # تغطية (Override)
        base = super().introduce()
        return f"{base}، أعمل كـ {self.job}"

emp = Employee("أحمد", 30, "مهندس", 5000)
print(emp.introduce())
```

الوراثة المتعددة (Multiple Inheritance)

```
class A:
    def method(self):
```

```
        return "Method A"

class B:
    def method(self):
        return "Method B"

class C(A, B):          # ترتيب الميراث مهم (MRO)
    pass

c = C()
print(c.method())      # "Method A" - حسب MRO
print(C.__mro__)       # ترتيب الوراثة
```

ديكوراتور @property

```
class Temperature:
    def __init__(self, celsius):
        self._celsius = celsius

    @property
    def fahrenheit(self):
        return (self._celsius * 9/5) + 32

    @property
    def celsius(self):
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            raise ValueError("!درجة الحرارة أقل من الصفر المطلق!")
        self._celsius = value

t = Temperature(25)
print(t.fahrenheit)    # 77.0
t.celsius = 30
print(t.fahrenheit)    # 86.0
```


18. المزخرفات (Decorators)

الديكوراتور هو دالة تأخذ دالة أخرى وتعديل سلوكها. تستخدم بكثرة في أطر العمل مثل Flask و Django.

ديكوراتور بسيط

```
def timer(func):
    def wrapper(*args, **kwargs):
        import time
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"استغرقت الدالة {end - start:.4f} ثانية")
        return result
    return wrapper

@timer
def slow_function():
    import time
    time.sleep(1)
    return "تم!"

slow_function()  # استغرقت الدالة 1.0001 ثانية
```

ديكوراتور بمعاملات

```
def repeat(n):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for _ in range(n):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator

@repeat(3)
def print_hello():
    print("Hello!")

print_hello()  # Hello! Hello! Hello!
```

ديكوراتورات مدمجة مفيدة

```
# @staticmethod - دالة ثابتة (لا تحتاج self)
# @classmethod - دالة الفئة (تستقبل cls بدلاً من self)
# @property - تحويل دالة إلى خاصية

class MathUtils:
    @staticmethod
    def add(a, b):
        return a + b

    @classmethod
```

```
def info(cls):  
    return f"هذه الفئة: {cls.__name__}"
```

```
print(MathUtils.add(3, 4))    # بدون إنشاء كائن - 7  
print(MathUtils.info())      # هذه الفئة: MathUtils
```

19. المولدات (Generators)

المولدات (Generators) هي دوال ترجع مكرر (Iterator) يولد القيم واحداً تلو الآخر. تستخدم `yield` بدلاً من `.return`.

```
# مولد بسيط
def count_up_to(n):
    count = 1
    while count <= n:
        yield count
        count += 1

# استخدام المولد
counter = count_up_to(5)
print(next(counter))    # 1
print(next(counter))    # 2

# التكرار على المولد
for num in count_up_to(3):
    print(num)           # 1, 2, 3
```

مولد أرقام فيبوناتشي

```
def fibonacci():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

fib = fibonacci()
for _ in range(10):
    print(next(fib))      # 0, 1, 1, 2, 3, 5, 8, 13, 21, 34
```

التعبير المولد (Generator Expression)

```
# List Comprehension (ينشئ القائمة كاملة في الذاكرة)
squares_list = [x**2 for x in range(10)]

# Generator Expression (يولد القيم عند الحاجة - يوفر للذاكرة)
squares_gen = (x**2 for x in range(10))

print(sum(squares_gen))  # 285
```

Comprehensions .20

comprehensions هي طريقة مختصرة وأنشئ لإنشاء القوائم، القواميس، والمجموعات.

List Comprehension

```
# الصيغة: [expression for item in iterable if condition]

# مربعات الأعداد من 0 إلى 9
squares = [x**2 for x in range(10)]
# [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# الأعداد الزوجية فقط
evens = [x for x in range(20) if x % 2 == 0]

# if-else مع شرط
labels = ["زوجي" if x % 2 == 0 else "فردى" for x in range(5)]
# ['زوجي', 'فردى', 'زوجي', 'فردى', 'زوجي']

# قوائم متداخلة
matrix = [[i*j for j in range(3)] for i in range(3)]
# [[0, 0, 0], [0, 1, 2], [0, 2, 4]]

# تسطيح قائمة متداخلة
nested = [[1, 2], [3, 4], [5, 6]]
flat = [item for sublist in nested for item in sublist]
# [1, 2, 3, 4, 5, 6]
```

Dict Comprehension

```
# الصيغة: {key: value for item in iterable if condition}

# قاموس بمربعات الأعداد
squares_dict = {x: x**2 for x in range(5)}
# {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}

# عكس المفتاح والقيمة
original = {"a": 1, "b": 2, "c": 3}
reversed_dict = {value: key for key, value in original.items()}
# {1: 'a', 2: 'b', 3: 'c'}

# تصفية القاموس
filtered = {k: v for k, v in original.items() if v > 1}
# {'b': 2, 'c': 3}
```

Set Comprehension

```
# الصيغة: {expression for item in iterable if condition}

# أطوال الكلمات (بدون تكرار)
words = ["hello", "world", "python", "code", "hello"]
lengths = {len(word) for word in words}
```


21. الدوال الجاهزة الهامة

الدالة	الوصف	مثال
<code>()print</code>	طباعة النص	<code>print("Hello")</code>
<code>()len</code>	طول الكائن	<code>len([1,2,3]) → 3</code>
<code>()type</code>	نوع الكائن	<code>type(10) → int</code>
<code>()int</code>	تحويل لعدد صحيح	<code>int("5") → 5</code>
<code>()str</code>	تحويل لنص	<code>"str(5) → "5</code>
<code>()list</code>	تحويل لقائمة	<code>list("abc") → ['a','b','c']</code>
<code>()dict</code>	إنشاء قاموس	<code>dict(a=1) → {'a': 1}</code>
<code>()range</code>	توليد سلسلة أعداد	<code>range(5) → 0..4</code>
<code>()enumerate</code>	ترقيم العناصر	<code>enumerate(['a','b'])</code>
<code>()zip</code>	دمج قائمتين	<code>zip([1,2], ['a','b'])</code>
<code>()map</code>	تطبيق دالة على كل عنصر	<code>map(str, [1,2,3])</code>
<code>()filter</code>	تصفية العناصر	<code>filter(is_even, nums)</code>
<code>()sorted</code>	ترتيب	<code>sorted([3,1,2])</code>
<code>()sum</code>	مجموع	<code>sum([1,2,3]) → 6</code>
<code>()max</code>	أقصى قيمة	<code>max(3,7,2) → 7</code>
<code>()min</code>	أدنى قيمة	<code>min(3,7,2) → 2</code>
<code>()abs</code>	القيمة المطلقة	<code>abs(-5) → 5</code>
<code>()all</code>	كل القيم True	<code>all([True, True]) → True</code>
<code>()any</code>	أي قيمة True	<code>any([False, True]) → True</code>

isinstance(5, int) → True

التحقق من نوع

()isinstance

أمثلة على دوال مفيدة

```
# enumerate() - ترقيم العناصر
names = ["أحمد", "سارة", "خالد"]
for i, name in enumerate(names, start=1):
    print(f"{i}. {name}")

# zip() - دمج قوائم
students = ["أحمد", "سارة", "خالد"]
grades = [85, 92, 78]
for name, grade in zip(students, grades):
    print(f"{name}: {grade}")

# zip() تفريغ (Unzip)
pairs = [("a", 1), ("b", 2), ("c", 3)]
letters, numbers = zip(*pairs)
print(letters)    # ('a', 'b', 'c')
print(numbers)    # (1, 2, 3)
```

مكتبات شائعة

المكتبة	الوصف	مجال الاستخدام
NumPy	عمليات رقمية ومصفوفات	الحسابات العلمية
Pandas	تحليل ومعالجة البيانات	علم البيانات
Matplotlib	رسم البيانات والرسوم البيانية	التصور البياني
Scikit-learn	تعلم آلي	ذكاء اصطناعي
TensorFlow / PyTorch	تعلم عميق	ذكاء اصطناعي
Flask / Django	تطوير الويب	ويب
Requests	إرسال طلبات HTTP	ويب / APIs
BeautifulSoup / Scrapy	استخراج بيانات المواقع	Web Scraping
PyGame	تطوير الألعاب	ألعاب
PyAutoGUI	التحكم بالفأرة ولوحة المفاتيح	أتمتة

أمثلة سريعة

```
# Requests - جلب بيانات من API
import requests
response = requests.get("https://api.github.com")
print(response.status_code) # 200
data = response.json()

# NumPy - مصفوفات رقمية
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
print(arr.mean()) # 3.0
print(arr.reshape(5, 1))

# Pandas - تحليل البيانات
import pandas as pd
df = pd.DataFrame({"name": ["أحمد", "سارة"], "age": [25, 30]})
print(df.describe())
```


23. أفضل الممارسات

كتابة كود نظيف

- استخدم أسماء معبرة للمتغيرات
- اتبع نمط snake_case للمتغيرات والدوال
- اتبع نمط PascalCase للفئات
- استخدم الثوابت بأحرف كبيرة $PI = 3.14$
- أضف تعليقات مفيدة للكود المعقد
- اكتب توثيق (Docstrings) للدوال والفئات

قواعد PEP 8 (أسلوب كتابة بايثون)

- استخدم 4 مسافات للمسافة البادئة (Indentation)
- الحد الأقصى لطول السطر: 79 حرفاً
- سطر فارغ بين الدوال
- سطرين فارغين بين الفئات
- مسافة بعد الفاصلة: `func(a, b)`
- مسافة حول العوامل: `x = 5`

نصائح للمبتدئين

1. ابدأ بمشاريع صغيرة وبسيطة
2. اكتب كوداً كل يوم حتى لو كان قليلاً
3. اقرأ كود الآخرين (مشاريع مفتوحة المصدر)
4. استخدم مصحح الأخطاء (Debugger)
5. تعلم Git لإدارة الكود
6. شارك في مجتمع بايثون
7. لا تحفظ الكود، افهم المنطق
8. استخدم virtualenv لإدارة المشاريع

مشاريع مقترحة للمبتدئين

- آلة حاسبة بسيطة
- مدير مهام (To-Do App)
- محول العملات
- لعبة تخمين الأرقام
- برنامج تذكير بالمهام
- محمّل فيديوهات من YouTube
- روبوت تويتر بسيط
- محلل نصوص (إحصائيات)

مثال متكامل: مدير مهام بسيط

```
# todo.py - مدير مهام بسيط

class TodoList:
    def __init__(self):
        self.tasks = []

    def add(self, task):
        self.tasks.append({"task": task, "done": False})
        print(f"✓ تمت إضافة {task}")

    def done(self, index):
        if 0 <= index < len(self.tasks):
            self.tasks[index]["done"] = True
            print(f"✓ تم إنجاز : {self.tasks[index]['task']}")

    def show(self):
        if not self.tasks:
            print("× توجد مهام")
            return
        for i, t in enumerate(self.tasks):
            status = "✓" if t["done"] else "○"
            print(f"{i}. {status} {t['task']}")

# الاستخدام
todo = TodoList()
todo.add("تعلم بايثون")
todo.add("قراءة كتاب")
todo.show()
```

```
todo.done(0)
todo.show()
```

مصادر تعليمية

- Python.org - الموقع الرسمي للغة
- W3Schools Python - دروس تفاعلية
- Real Python - مقالات ودروس متقدمة
- Python for Everybody - دورة مجانية
- Automate the Boring Stuff with Python - كتاب مجاني
- LeetCode - حل مشاكل برمجية
- GitHub - استكشاف مشاريع مفتوحة المصدر

دليل لغة بايثون - Python Programming Guide v1.0 | 2026

تم إعداد هذا الدليل كمرجع شامل لتعلم لغة البرمجة بايثون